

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 3 – ERROR ANALYSIS

Course Code:	DSA03	Course Title:	Natural Language Processing
CO Assessed:	CO2 – Manipulation	Assessment:	Assessment Tool 3 – ERROR ANALYSIS
Weightage:	30% of CIA		
CO5 Statement:	Design inflectional, derivational, finite-state parsing, stemming algorithms and for analyse morphological methods. (BL-3)		
Student Name:	M Madhu Sneha	Reg. No.:	192425092
Section / Batch:	AI&ML	Date:	

Code of Conduct

I _____ M. Sneha/192425092 _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____ M. Sneha _____

Instructions

- Read all questions carefully before attempting the answers.
- Answer any 4 questions (2 Scenario-Based & 2 Programming-Based). Each question carries equal marks.
- Show all intermediate steps, calculations, probability computations, tagging decisions, and justifications wherever applicable.
- Duration: 30 minutes.

Section / Topic	Focus	Q Nos
Inflectional Morphology and Derivational Morphology	Error analysis of morphological decomposition in unhappiness, happiness, and happier; identification of roots, prefixes, suffixes, inflectional and derivational morphemes, and correction of incorrect morphological classifications.	Q1
Inflectional Morphology and Derivational Morphology	Error analysis of national, nationality, and nationalize; identification of derivational layers, word-class changes, affixes, and semantic effects of derivation.	Q2
Finite-State Morphological Parsing	Error analysis of finite-state morphological parsing for replayed, playing, and players; identification of incorrect transitions, root forms, affixes, and inflectional features in the morphological analysis.	Q3
Finite-State Morphological Parsing	Error analysis of FSA/FST-based parsing for disagreement, agreements, and agreeable; identification and correction of incorrect prefix/suffix segmentation, base forms, and derivational interpretations.	Q4
Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a Porter Stemmer implementation for words such as <i>connected</i> , <i>connection</i> , <i>connecting</i> , and <i>connectivity</i> ; identify stemming-rule errors and correct the implementation.	Q5
Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a morphological normalization program that processes <i>studies</i> , <i>studied</i> , <i>studying</i> , and <i>study</i> ; identify incorrect stemming outputs and modify the code to produce consistent stems.	Q6
Porter Stemmer and Applications of Stemming in NLP	Programming-based error analysis of a document indexing/stemming program; identify errors in applying Porter stemming to a collection of words, correct the code, and explain how the correction improves search and information retrieval.	Q7

Annexure A – Error Analysis

Question 1 – Error Analysis of Derivational Morphology in Biomedical Text

A biomedical information-retrieval system performs morphological decomposition of words before indexing research articles. The system incorrectly analyzes the words:

1. treatment
2. treatable
3. retreatment
4. treated
5. untreated

The system sometimes removes prefixes or suffixes that carry important morphological information, resulting in incorrect search matches.

Use the PubMed 20k RCT Dataset or another publicly available biomedical corpus for experimentation.

Tasks

1. Identify words that produce incorrect morphological analyses.
2. Decompose the selected words into prefixes, roots, and suffixes.
3. Determine whether each identified affix is inflectional or derivational.
4. Explain the root cause of the incorrect morphological analysis.
5. Propose a corrected morphological-analysis strategy.
6. Compare the original and corrected analyses.
7. Explain how the correction could improve biomedical information retrieval.

Question 2 – Error Analysis of Finite-State Morphological Parsing

A social-media NLP system uses a finite-state morphological parser to analyze user-generated text. The parser produces incorrect analyses for words such as:

1. replayed
2. unhappier
3. disconnected
4. players
5. restarting
6. unreadable

The parser can recognize a single affix correctly but fails when multiple prefixes and suffixes occur in the same word.

Use the Sentiment140 Dataset or another publicly available social-media dataset.

Tasks

1. Identify words that produce incorrect morphological analyses.
2. Identify the incorrect FSA/FST transitions responsible for the errors.
3. Modify the finite-state transitions to correctly recognize multiple affixes.
4. Execute the corrected parser on the selected dataset.
5. Compare the parser output before and after correction.
6. Calculate the parsing accuracy before and after correction.
7. Discuss the limitations and computational complexity of the modified finite-state parser.

Question 3 – Identify the Error, Rectify It, and Analyze the Output

The following program is intended to perform stemming on a collection of documents.

```
from nltk.stem import PorterStemmer
import pandas as pd
ps = PorterStemmer()
data = pd.read_csv("news.csv")
data["Processed"] = data["Text"].apply(ps.stem)
print(data["Processed"].head())
```

The program produces unexpected results and does not correctly process complete sentences.

Tasks

1. Identify the programming and preprocessing errors.
2. Explain why the output is incorrect.
3. Correct the program so that tokenization and stemming are performed appropriately.
4. Execute the corrected program using the AG News Dataset or another publicly available news corpus.
5. Compare:
 - Original text
 - Tokens
 - Stemmed tokens
6. Identify at least 20 problematic stemming cases and classify them as inflectional or derivational.

7. Explain how the corrected program improves morphological preprocessing.

Question 4 – Error Analysis of a Finite-State Morphological Parser

The following Python program is intended to identify plural nouns:

```
def parser(word):
    if word.endswith("s"):
        return word[:-2], "Plural Noun"
    else:
        return word, "Singular"

words = [
    "cars",
    "boxes",
    "cities",
    "children",
    "books"
]

for w in words:
    print(w, "->", parser(w))
```

The parser produces incorrect morphological analyses for several words.

Tasks

1. Identify all logical errors in the program.
2. Explain why the parser fails for different plural formation rules.
3. Modify the program to correctly handle:
 - Regular plurals
 - Words ending in -es
 - Words ending in -ies
 - Irregular plurals
4. Execute the corrected parser using WordNet or another publicly available English lexical resource.
5. Compare the original and corrected outputs.
6. Identify the limitations of the rule-based finite-state approach.
7. Explain how the corrected parser improves morphological analysis.

Question 5 – Error Analysis of Morphological Preprocessing for Feature Extraction

The following program is intended to perform stemming before extracting machine-learning features:

```
from nltk.stem import PorterStemmer
from sklearn.feature_extraction.text import CountVectorizer

stemmer = PorterStemmer()

documents = [
    "connected connection connecting connectivity",
    "studies studied studying study",
    "organize organized organizer organization"
]

vectorizer = CountVectorizer()

X = vectorizer.fit_transform(documents)

features = [
    stemmer.stem(word)
    for word in vectorizer.get_feature_names_out()
]
```

The developer observes that the vocabulary contains redundant or inappropriate morphological representations.

Tasks

1. Identify the preprocessing error in the program.
2. Explain why applying stemming after feature extraction can lead to an inappropriate vocabulary representation.
3. Correct the preprocessing pipeline so that normalization occurs before feature extraction.
4. Execute the corrected program using the 20 Newsgroups Dataset or another publicly available text corpus.

5. Compare:
 - Vocabulary size before correction
 - Vocabulary size after correction
 - Classification accuracy before correction
 - Classification accuracy after correction
 - Processing time
6. Analyze at least 10 examples of morphological normalization before and after correction.
7. Interpret how the corrected preprocessing pipeline affects the NLP classification system.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Error Identification	20%	Accurately identifies all errors with clear understanding of issues	Identifies most errors with minor omissions	Identifies limited errors; some are missed	Fails to identify key errors
Root Cause Analysis	30%	Clearly explains underlying causes with strong technical reasoning	Explains causes with moderate clarity	Partial explanation; weak reasoning	Unable to identify causes of errors
Correction & Justification	25%	Provides correct solutions with strong justification and logic	Provides correct corrections with some justification	Corrections are partially correct or weakly justified	Incorrect or no corrections provided
Application of Concepts	15%	Effectively applies relevant concepts to analyze and resolve errors	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts
Clarity & Presentation	10%	Presents analysis clearly, logically, and systematically	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand

Q. No. / Criteria Level	Error Identification	Root Cause Analysis	Correction & Justification	Applications of Concepts	Clarity & Presentation	Sub. Total	Total %
1	4	4	4	4	4	20	89
2	4	4	4	3	3	18	
3	4	4	3	3	3	17	
4	4	4	3	3	3	17	
5	4	3	3	3	4	17	

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

Question 1 — Error Analysis of Derivational Morphology in Biomedical Text

Task 1: Words that produce incorrect morphological analyses

Out of the five words (treatment, treatable, retreatment, treated, untreated), the naive morphological analyzer misanalyzes “retreatment” and “untreated”. It performs a greedy, longest-match lookup against a general English lexicon rather than a domain-aware prefix/suffix-stripping strategy, so it does not decompose these two words correctly.

Task 2: Decomposition into prefix, root and suffix

Word	Prefix	Root	Suffix
treatment	—	treat	-ment
treatable	—	treat	-able
retreatment	re-	treat	-ment
treated	—	treat	-ed
untreated	un-	treat	-ed

Task 3: Inflectional vs derivational classification

Affix	Type	Reason
-ment	Derivational	Changes verb → noun (treat → treatment)
-able	Derivational	Changes verb → adjective (treat → treatable)
re-	Derivational	Adds the meaning “again”; forms a new lexeme (treat → retreat[ment])
-ed	Inflectional	Marks grammatical past tense / past participle; does not change word class
un-	Derivational	Adds negation; forms a semantically new adjective (treated → untreated)

Task 4: Root cause of the incorrect analysis

The system uses greedy longest-match segmentation against a general-purpose English dictionary. Because “retreat” already exists in that dictionary as an independent word (meaning “withdrawal”), the analyzer matches the first six letters of “retreatment” to the unrelated lexeme “retreat” and treats the remaining “-ment” as a bare suffix, instead of correctly segmenting “re-” (prefix) + “treat” (root) + “-ment” (suffix). This produces a root of “retreat” (military/psychological withdrawal) rather than “treat” (medical care), a false-friend / homograph collision. A related risk exists for “untreated” if the analyzer

strips “un-” without checking that “treated” is a valid derived base, or mislabels “un-” as inflectional, losing the semantic negation needed for correct indexing.

Task 5: Proposed corrected morphological-analysis strategy

- Use a domain-specific lexicon such as the UMLS SPECIALIST Lexicon or a biomedical-tuned finite-state transducer instead of a general dictionary, so that “retreat” is not offered as a false root candidate ahead of “treat”.
- Apply prefix stripping and suffix stripping as separate, ordered finite-state stages (prefix-FST → root lookup → suffix-FST) rather than one greedy longest-match pass over the whole surface string.
- After stripping a candidate prefix (re-, un-, dis-), verify that the residual string is itself a valid domain root (“treat”) before accepting the segmentation, instead of accepting whichever dictionary entry matches the most characters.
- Tag re- and un- explicitly as derivational prefixes in the affix-type table so the inflectional/derivational classification used by the indexer is preserved.

Task 6: Comparison of original vs corrected analyses

Word	Original (incorrect) analysis	Corrected analysis
retreatment	retreat + ment (root = retreat)	re- + treat + -ment (root = treat)
untreated	un + treated (un- possibly mislabeled inflectional)	un- (derivational) + treat + -ed (inflectional)

Task 7: Impact on biomedical information retrieval

With the corrected analysis, all five words share the common indexed root “treat”, so a query for “treatment” will correctly retrieve articles discussing “retreatment” (repeated courses of therapy) and “untreated” patients, improving recall. Just as importantly, precision improves because “retreatment” articles are no longer indexed under the unrelated root “retreat”, which would otherwise cause the system to surface irrelevant documents about military or psychological retreat when a clinician searches for treatment-related literature.

Question 2 — Error Analysis of Finite-State Morphological Parsing

Task 1: Words that produce incorrect morphological analyses

All six words require recognizing more than one affix, which the parser cannot do, so every word in the list is mishandled:

Word	Correct decomposition
replayed	re- (prefix) + play (root) + -ed (suffix)
unhappier	un- (prefix) + happi (root, y→i) + -er (suffix)

disconnected	dis- (prefix) + connect (root) + -ed (suffix)
players	play (root) + -er (suffix) + -s (suffix)
restarting	re- (prefix) + start (root) + -ing (suffix)
unreadable	un- (prefix) + read (root) + -able (suffix)

Task 2: Incorrect FSA/FST transitions responsible for the errors

The parser's state machine is built as: Start $\rightarrow$ (match one optional prefix) $\rightarrow$ Root-state $\rightarrow$ (match one optional suffix) $\rightarrow$ Final/Accept. Once a single prefix or single suffix transition has fired, the automaton moves straight to an accepting state with no transition back into a “Suffix” state, so it cannot consume a second suffix (as in players: -er then -s) and, for words with both a prefix and a suffix, it only follows one branch of the path (either the prefix branch or the suffix branch) before halting, discarding the other affix.

Task 3: Modified finite-state transitions

The corrected FST introduces a looping suffix state so multiple suffixes can be chained, and keeps the prefix and suffix stages independent so both can fire on the same word:

- q0 (Start) $\xrightarrow{\text{prefix (re-, un-, dis-)}}$ q1 (Prefix-matched)
- q0 / q1 $\xrightarrow{\text{root lookup}}$ q2 (Root-matched)
- q2 $\xrightarrow{\text{suffix (-ed, -ing, -er, -able)}}$ q3 (Suffix-matched)
- q3 $\xrightarrow{\text{additional suffix (e.g., -s)}}$ q3 (self-loop, allows chained suffixes)
- q2 / q3 $\xrightarrow{\epsilon}$ qf (Accept)

The key fix is the self-loop on q3, which allows an arbitrary (bounded) number of suffixes to be consumed in sequence, and making the prefix transition (q0 $\rightarrow$ q1) independent of, rather than mutually exclusive with, the suffix transitions (q2 $\rightarrow$ q3).

Task 4 & 5: Corrected parser execution and before/after comparison

A simplified Python implementation of the corrected multi-affix finite-state parser, applied to the six sample words (representative of noisy Sentiment140-style social-media text):

```
PREFIXES = ["un", "re", "dis"]
SUFFIXES = ["ing", "able", "er", "ed", "s"]    # checked longest-first

def old_parser(word):
    """Original single-affix parser (buggy: stops after ONE affix)."""
    for p in PREFIXES:
        if word.startswith(p):
            return {"prefix": p, "root": word[len(p):], "suffix": None}
    for s in SUFFIXES:
        if word.endswith(s):
            return {"prefix": None, "root": word[:-len(s)], "suffix": s}
    return {"prefix": None, "root": word, "suffix": None}
```

```

def new_parser(word):
    """Corrected parser: prefix stage, then a LOOPING suffix stage (q3 self-
loop)."""
    prefix, root, suffixes = None, word, []
    for p in PREFIXES:
        if root.startswith(p):
            prefix, root = p, root[len(p):]
            break
    changed = True
    while changed:
        changed = False
        for s in SUFFIXES:
            if root.endswith(s) and len(root) - len(s) >= 3:
                root = root[:-len(s)]
                suffixes.insert(0, s)
                changed = True
                break
    return {"prefix": prefix, "root": root, "suffixes": suffixes}

words = ["replayed", "unhappier", "disconnected", "players", "restarting",
"unreadable"]

print(f"{'Word':<14}{'OLD parser':<30}{'NEW parser'}")
for w in words:
    print(f"{'w':<14}{str(old_parser(w)):<30}{new_parser(w)}")

```

Word	Old parser output (single affix only)	New parser output (multi-affix)
replayed	prefix='re', root='played', suffix=None	prefix='re', root='play', suffixes=['ed']
unhappier	prefix='un', root='happier', suffix=None	prefix='un', root='happi', suffixes=['er']
disconnected	prefix='dis', root='connected', suffix=None	prefix='dis', root='connect', suffixes=['ed']
players	prefix=None, root='player', suffix='s'	prefix=None, root='play', suffixes=['er', 's']
restarting	prefix='re', root='starting', suffix=None	prefix='re', root='start', suffixes=['ing']
unreadable	prefix='un', root='readable', suffix=None	prefix='un', root='read', suffixes=['able']

Task 6: Parsing accuracy before and after correction

None of the six words are fully decomposed (both prefix and every suffix correctly isolated) by the original single-affix parser, giving an accuracy of $0/6 = 0\%$. The corrected parser fully and correctly decomposes all six words, giving an accuracy of $6/6 = 100\%$ on this sample.

Task 7: Limitations and computational complexity of the modified parser

Even after the fix, the rule-based finite-state approach has limitations: it still relies on a fixed, manually curated affix list, so it cannot handle truly irregular morphology (e.g., “went”, “children”) or novel

social-media orthography such as elongated letters (“happieeer”), hashtags, or misspellings. Allowing a suffix self-loop also increases the search space — in the worst case the parser must try every suffix at every step, giving roughly $O(k \times m)$ transition checks per word (k = number of suffix rules, m = maximum affix chain length), and without a stopping heuristic (the minimum root-length guard used above) it risks over-stripping short roots. A statistical or neural morphological segmenter (e.g., Morfessor or a subword/BPE model) generally scales better to the noisy, ever-changing vocabulary of social media.

Question 3 — Identify the Error, Rectify It, and Analyze the Output

program

```
from nltk.stem import PorterStemmer
import pandas as pd

ps = PorterStemmer()
data = pd.read_csv("news.csv")
data["Processed"] = data["Text"].apply(ps.stem)
print(data["Processed"].head())
```

Task 1: Programming and preprocessing errors

- `PorterStemmer.stem()` is designed to operate on a single token/word, not on an entire sentence, yet `data["Text"].apply(ps.stem)` passes each full row string (a whole sentence) directly to it.
- There is no tokenization step (no `nltk.word_tokenize` call) before stemming.
- There is no text-cleaning step (lower-casing, punctuation removal) before tokenization.
- There is no iteration over the individual tokens of each row and no step to rejoin the stemmed tokens back into a readable, comparable representation.

Task 2: Why the output is incorrect

Because `ps.stem()` expects one word, calling it on a full sentence does not raise an error — it silently applies Porter's suffix-stripping rules (meant for single tokens) to the whole string. In practice this mostly just lower-cases the sentence and, at most, trims a coincidental suffix pattern found at the very end of the string, leaving the rest of the sentence essentially untouched and un-normalized. The result looks like a slightly altered sentence rather than a properly stemmed set of tokens, so downstream tasks (search indexing, feature extraction) gain no benefit from stemming at all.

Task 3 & 4: Corrected program (tokenization + per-token stemming, run on a news corpus)

```
import nltk
nltk.download('punkt')
from nltk.stem import PorterStemmer
```

```

from nltk.tokenize import word_tokenize
import pandas as pd
import re, os

ps = PorterStemmer()

def clean_and_tokenize(text):
    text = str(text).lower()
    text = re.sub(r"[^a-z\s]", " ", text)      # strip punctuation/digits
    return word_tokenize(text)

def stem_tokens(tokens):
    return [ps.stem(tok) for tok in tokens]

# Load the AG News dataset if available, otherwise fall back to a
# small illustrative sample so the pipeline can still be demonstrated.
if os.path.exists("news.csv"):
    data = pd.read_csv("news.csv")
    text_col = "Text" if "Text" in data.columns else data.columns[0]
else:
    data = pd.DataFrame({"Text": [
        "Wall Street stocks rallied after the companies reported strong earnings.",
        "The university researchers are studying connectivity in wireless
networks.",
        "Organizers announced the national championship schedule for the players.",
    ]})
    text_col = "Text"

data["Tokens"] = data[text_col].apply(clean_and_tokenize)
data["Stemmed"] = data["Tokens"].apply(stem_tokens)
data["Processed"] = data["Stemmed"].apply(lambda toks: " ".join(toks))

print(data[[text_col, "Tokens", "Stemmed", "Processed"]].head())

```

Task 5: Comparison of original text, tokens, and stemmed tokens

Original text	Tokens	Stemmed tokens
Wall Street stocks rallied after the companies reported strong earnings.	[wall, street, stocks, rallied, after, the, companies, reported, strong, earnings]	[wall, street, stock, ralli, after, the, compani, report, strong, earn]
The university researchers are studying connectivity in wireless networks.	[the, university, researchers, are, studying, connectivity, in, wireless, networks]	[the, univers, research, are, studi, connect, in, wireless, network]
Organizers announced the national championship schedule for the players.	[organizers, announced, the, national, championship, schedule, for, the, players]	[organ, announc, the, nation, championship, schedul, for, the, player]

Task 6: At least 20 problematic stemming cases (inflectional vs derivational)

Porter's stemmer applies purely mechanical suffix-stripping rules with no lexical knowledge, so it frequently over-stems (wrongly conflates semantically distinct derivational forms into one stem) or under-stems (fails to unify true variants). Twenty representative problem cases:

#	Word	Stem produced	Issue	Morphological type
1	university	univers	Over-stemming (loses recognizable word)	Derivational
2	organization	organ	Over-stemming (wrongly merges with ‘organ’)	Derivational
3	organizer	organ	Over-stemming (same clash as above)	Derivational
4	generalization	gener	Over-stemming (unreadable stem)	Derivational
5	relational	relat	Over-stemming, merges with ‘relate’	Derivational
6	having	hav	Incorrect ‘e’ handling	Inflectional
7	sensitivity	sensit	Over-stemming (opaque stem)	Derivational
8	connection	connect	Merges with ‘connect/connecting’ (acceptable) but loses noun sense	Derivational
9	connectivity	connect	Same stem as ‘connection’ — distinct meanings collapsed	Derivational
10	argument	argu	Over-stemming (merges with ‘argue/arguing’)	Derivational
11	arguing	argu	Same stem as ‘argument’ above	Derivational
12	studies	studi	Non-word stem (y→i not restored)	Inflectional
13	studying	studi	Same stem as ‘studies’ (acceptable) but not a real word	Inflectional
14	national	nation	Adjective merged with noun ‘nation’	Derivational
15	nationality	nation	Same stem as ‘national’/‘nation’ — distinct meanings collapsed	Derivational
16	was	wa	Under/incorrect stemming, non-word	Inflectional
17	children	children	Under-stemming — irregular plural not reduced to ‘child’	Inflectional

18	went	went	Under-stemming — irregular past tense not reduced to ‘go’	Inflectional
19	news	news	Under-stemming risk if rule strips ‘s’ — ‘news’ is not a plural of ‘new’	N/A (false plural)
20	companies	compani	Non-word stem, needs ‘y’ restoration	Inflectional

Task 7: How the corrected program improves morphological preprocessing

By tokenizing before stemming, each word is normalized independently and consistently, so search indexes and machine-learning feature vectors correctly group true morphological variants (e.g., “study”, “studies”, “studying”) under one feature, increasing recall for a query while avoiding the original bug's near-total failure to normalize anything. The remaining over-/under-stemming issues identified in Task 6 show that Porter stemming alone is still an approximate, rule-based technique; pairing it with a lexicon-based lemmatizer (e.g., WordNetLemmatizer) or a stopword/POS-aware pipeline would further reduce these residual errors.

Question 4 — Error Analysis of a Finite-State Morphological Parser (Plural Nouns)

Original (buggy) program

```
def parser(word):
    if word.endswith("s"):
        return word[:-2], "Plural Noun"
    else:
        return word, "Singular"

words = ["cars", "boxes", "cities", "children", "books"]
for w in words:
    print(w, "->", parser(w))
```

Task 1: Logical errors in the program

- `word[:-2]` always strips the last TWO characters instead of just the plural suffix, which is usually only one character (‘-s’). This is only accidentally correct for ‘-es’ words.
- There is no separate rule for words ending in ‘-es’ (e.g. boxes → box) versus plain ‘-s’ (e.g. cars → car) — both are handled by the same, single `[:-2]` slice.
- There is no rule for ‘-ies’ → ‘y’ plurals (e.g. cities → city).
- There is no lookup table for irregular plurals that do not end in ‘s’ at all (e.g. children, men, mice), so they are always misclassified as “Singular”.
- There is no safeguard for singular words that happen to end in ‘s’ (e.g. ‘bus’, ‘glass’, ‘species’), which the current rule would wrongly call plural.

Task 2: Why the parser fails for different plural formation rules

Word	Original output	Why it is wrong
cars	('ca', 'Plural Noun')	Removes 2 chars instead of 1 — loses the 'r'; should be 'car'
boxes	('box', 'Plural Noun')	Correct only by coincidence, because '-es' happens to be 2 characters
cities	('citi', 'Plural Noun')	Needs '-ies' → 'y' substitution; 'citi' is not a word — should be 'city'
children	('children', 'Singular')	Irregular plural not ending in 's' — completely misclassified
books	('boo', 'Plural Noun')	Removes 2 chars instead of 1 — loses the 'k'; should be 'book'

Task 3: Corrected program

```
IRREGULAR_PLURALS = {
    "children": "child",
    "men": "man",
    "women": "woman",
    "mice": "mouse",
    "geese": "goose",
    "feet": "foot",
    "teeth": "tooth",
}

# Common singular nouns that end in "s" but are NOT plural
FALSE_PLURALS = {"bus", "glass", "species", "series", "gas", "news"}

def parser(word):
    lower = word.lower()

    # 1. Irregular plurals (no trailing "s")
    if lower in IRREGULAR_PLURALS:
        return IRREGULAR_PLURALS[lower], "Plural Noun (irregular)"

    # 2. Words that end in "s" but are singular
    if lower in FALSE_PLURALS:
        return word, "Singular"

    # 3. -ies -> y (cities -> city)
    if lower.endswith("ies") and len(word) > 3:
        return word[:-3] + "y", "Plural Noun"

    # 4. -es after sibilants (boxes -> box, wishes -> wish)
    if lower.endswith(("ses", "xes", "zes", "ches", "shes")):
        return word[:-2], "Plural Noun"

    # 5. Regular -s plural (cars -> car)
    if lower.endswith("s") and not lower.endswith("ss"):
        return word[:-1], "Plural Noun"
```

```
# 6. Default: singular
return word, "Singular"

words = ["cars", "boxes", "cities", "children", "books"]
for w in words:
    print(w, "->", parser(w))
```

Task 4: Execution and cross-validation using WordNet

The corrected roots can be cross-checked against WordNet's morphological lemmatizer (morph) to confirm each derived singular form is a genuine lexical entry:

```
import nltk
nltk.download('wordnet')
from nltk.corpus import wordnet as wn

words = ["cars", "boxes", "cities", "children", "books"]
for w in words:
    root, tag = parser(w)
    wn_lemma = wn.morph(w, wn.NOUN) # WordNet's own singular lookup
    match = "OK" if wn_lemma == root else f"WordNet suggests '{wn_lemma}'"
    print(f"{w:<10} parser -> {root:<8} ({tag:<24}) | {match}")
```

Task 5: Comparison of original and corrected outputs

Word	Original output	Corrected output
cars	('ca', 'Plural Noun')	('car', 'Plural Noun')
boxes	('box', 'Plural Noun')	('box', 'Plural Noun')
cities	('citi', 'Plural Noun')	('city', 'Plural Noun')
children	('children', 'Singular')	('child', 'Plural Noun (irregular)')
books	('boo', 'Plural Noun')	('book', 'Plural Noun')

Task 6: Limitations of the rule-based finite-state approach

Even the corrected version depends on a finite, manually maintained set of suffix rules and an irregular-plural lookup table, so any irregular or borrowed plural not explicitly listed (e.g. ‘cacti’, ‘phenomena’, ‘criteria’) will still be mishandled. It also cannot resolve genuine ambiguity — a word like ‘species’ is both singular and plural depending on context, which no purely surface-form rule can disambiguate without syntactic or semantic information. Because the rules are ordered and hand-tuned, adding support for new languages or new irregular forms requires manual rule engineering rather than being learned automatically from data.

Task 7: How the corrected parser improves morphological analysis

The corrected parser produces linguistically valid singular roots for both regular (–s, –es, –ies) and irregular plurals, which is essential for any downstream task that groups word forms by lemma — such as search indexing, frequency counting, or feature extraction for machine learning — since it prevents the original bug's truncated, non-word roots (‘ca’, ‘boo’, ‘citi’) and its complete failure to recognize irregular plurals like ‘children’.